

DOSSIER DE PROJET

Titre professionnel
Développeur Web et Web Mobile

INSTANT TENNIS

API REST de gestion de joueurs, tournois et inscriptions

Salah Belhassan

Composante backend — Dossier de conception et de développement

Eureka / LaPlateforme

SOMMAIRE

01	Présentation du projet.....	3
02	Environnement technique et technologies utilisées	4
03	Architecture de l'application	5
04	Synthèse des compétences professionnelles mises en œuvre	7
05	CP5 — Mettre en place une base de données relationnelle	8
06	CP6 — Développer des composants d'accès aux données SQL et NoSQL.....	12
07	CP7 — Développer des composants métier côté serveur.....	14
08	CP8 — Documenter le déploiement d'une application web.....	18
09	Journalisation et supervision	21
10	Plan de tests, jeu d'essai et couverture de code.....	22
11	Microservices : architecture cible	25
12	Perspectives d'évolution.....	27
13	Conclusion.....	29

01 PRÉSENTATION DU PROJET

Instant Tennis est une API REST développée avec Spring Boot dont l'objectif est de gérer le classement de joueurs de tennis, l'organisation de tournois et l'inscription des joueurs à ces tournois. Ce dossier documente exclusivement la partie backend de l'application : conception de la base de données, architecture logicielle, développement des couches Web, Métier et Données, sécurité applicative, journalisation, conteneurisation et plan de tests.

Contexte fonctionnel

L'application expose une API REST sécurisée qui permet :

- de consulter, créer, modifier et supprimer des joueurs (Player) ;
- de consulter, créer, modifier et supprimer des tournois (Tournament) ;
- d'inscrire un joueur à un tournoi, avec vérification de la capacité disponible et de l'absence de doublon d'inscription ;
- de calculer et de recalculer automatiquement le classement (rank) des joueurs à chaque création, modification ou suppression d'un joueur, en fonction de ses points ;
- de gérer l'authentification et l'autorisation des utilisateurs (rôles ROLE_USER et ROLE_ADMIN) via un mécanisme de session sécurisé.

Périmètre du présent dossier

Conformément aux consignes du formateur, ce dossier se concentre uniquement sur la conception et le développement du backend de l'application, c'est-à-dire l'activité type « Développer la partie back-end d'une application web ou web mobile sécurisée ». Il ne traite pas d'interface graphique : l'application est consommée via des clients REST (Postman, Swagger UI) ou par un futur frontend qui n'entre pas dans le périmètre documenté ici.

La structure de ce dossier suit directement les quatre compétences professionnelles rattachées à cette activité type, afin de faciliter leur évaluation :

- CP5 — Mettre en place une base de données relationnelle ;
- CP6 — Développer des composants d'accès aux données SQL et NoSQL ;
- CP7 — Développer des composants métier côté serveur ;
- CP8 — Documenter le déploiement d'une application dynamique web ou web mobile.

02 ENVIRONNEMENT TECHNIQUE ET TECHNOLOGIES UTILISÉES

L'application repose sur un environnement technique moderne construit autour de Java 25 et du framework Spring Boot (version 4.0.0), qui offre un écosystème complet facilitant la configuration et le développement d'une API REST sécurisée.

Spring Boot : les apports clés du framework

Serveur web embarqué

Spring Boot intègre un serveur Tomcat embarqué, ce qui permet d'exécuter l'application de façon autonome (via `java -jar`) sans configuration externe ni déploiement manuel sur un serveur d'application.

Gestion simplifiée des composants avec les annotations

Les annotations telles que `@RestController`, `@Service` et `@Repository` permettent au conteneur Spring de détecter, instancier et injecter automatiquement les beans nécessaires au démarrage de l'application, réduisant le couplage entre les composants.

Accès aux données simplifié avec Spring Data JPA

Spring Data JPA, associé à Hibernate, simplifie les opérations CRUD et la communication avec la base de données relationnelle. Grâce au mécanisme d'ORM (Object Relational Mapping), les données sont manipulées sous forme d'objets Java (entités) sans écrire directement de requêtes SQL pour les cas standards.

Le rôle de Maven

Maven gère le cycle de vie du projet, ses dépendances et sa compilation. Le fichier `pom.xml` centralise toutes les bibliothèques utilisées et définit deux profils Maven (dev et prod) qui activent des jeux de propriétés Spring différents selon l'environnement cible, garantissant un build reproductible.

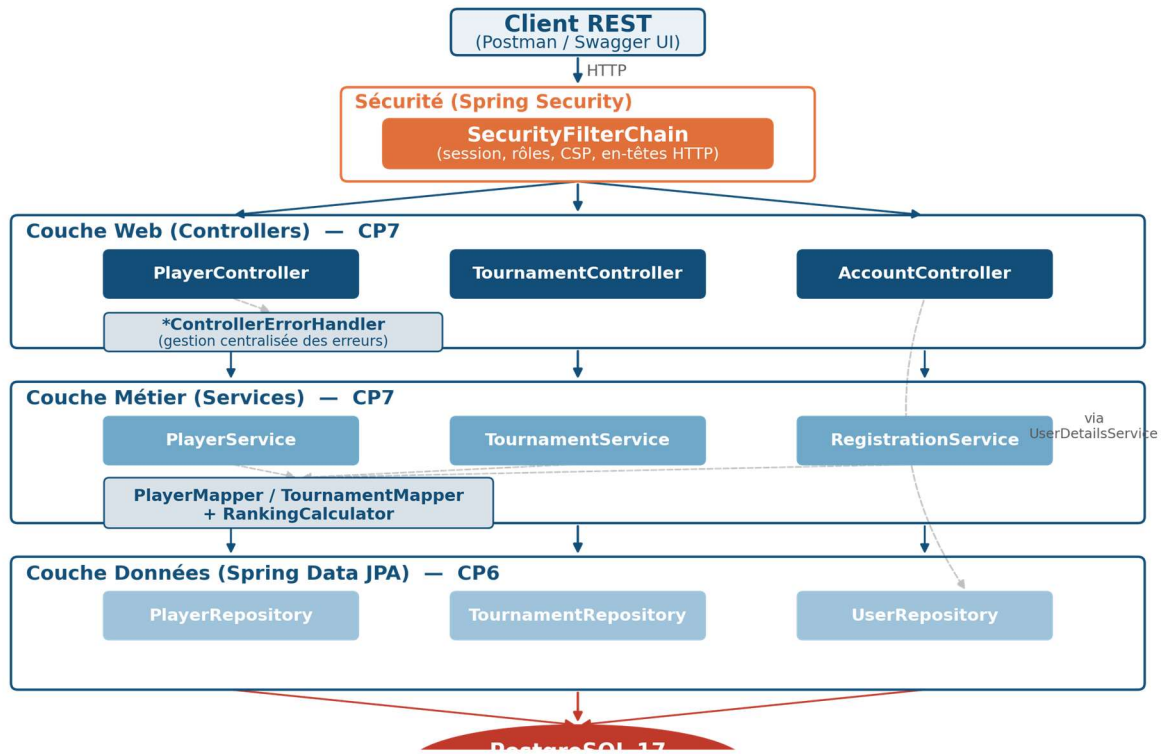
Technologies utilisées

Catégorie	Technologies
Langage	Java 25
Framework applicatif	Spring Boot 4.0.0 (Web, Data JPA, Validation, Security, Actuator)
Base de données	PostgreSQL 17 (H2 en mémoire pour les tests)
Migrations de schéma	Flyway (spring-boot-starter-flyway)
Sécurité	Spring Security (session, BCrypt, en-têtes HTTP)
Documentation d'API	springdoc-openapi (Swagger UI / OpenAPI 3)
Journalisation	SLF4J / Logback, logback-gelf (export GELF vers Graylog)
Observabilité requêtes SQL	datasource-proxy (traçage des requêtes générées par Hibernate)
Supervision	Spring Boot Actuator (endpoints health, metrics)
Gestion des dépendances	Maven (pom.xml, profils dev / prod)
Conteneurisation	Docker (Dockerfile multi-stage), Docker Compose
Tests et couverture	JUnit 5, Mockito, AssertJ, MockMvcTester, TestRestTemplate, JaCoCo
IDE / Outils	IntelliJ IDEA, Git / GitHub, Postman

La séparation des profils Maven dev / prod (activés par `-Pdev` ou `-Pprod`) permet de charger des propriétés de connexion et des jeux de migrations Flyway différents sans dupliquer le code, ce qui respecte le principe de configuration externalisée recommandé pour les applications 12-factor.

03 ARCHITECTURE DE L'APPLICATION

Instant Tennis suit une architecture logicielle en couches (layered architecture), organisée en un seul module Maven mais avec une séparation stricte des responsabilités entre trois packages principaux : web (présentation), service (métier) et data (accès aux données), complétés par un package model (DTOs exposés par l'API) et un package security (authentification / autorisation).



Architecture en couches de Instant Tennis

Description des couches

Couche	Rôle	Composants	Compétence
Web (présentation)	Expose les endpoints REST, valide les entrées et traduit les exceptions métier en réponses HTTP normalisées.	PlayerController, TournamentController, AccountController, *ControllerErrorHandler	CP7
Service (métier)	Applique les règles de gestion, orchestre les appels aux repositories et transforme les entités en DTOs exposables.	PlayerService, TournamentService, RegistrationService, PlayerMapper, TournamentMapper, RankingCalculator	CP7
Data (persistance)	Encapsule l'accès à la base de données via Spring Data JPA.	PlayerRepository, TournamentRepository, UserRepository (interfaces JpaRepository)	CP6
Security	Configure l'authentification, les autorisations par rôle et les en-têtes de sécurité HTTP.	SecurityConfiguration, DymaUserDetailsService	CP7

La dernière colonne du tableau ci-dessus renvoie à la compétence professionnelle backend illustrée par chaque couche ; ce rapprochement est développé en détail dans les quatre chapitres suivants (CP5 à CP8).

Design patterns et principes appliqués

Séparation DTO / Entité (Data Transfer Object)

L'API n'expose jamais directement les entités JPA (PlayerEntity, TournamentEntity) : elle communique via des records Java immuables (Player, PlayerToCreate, PlayerToUpdate, Tournament...) définis dans le package model. Cette séparation évite de coupler le contrat d'API à la structure interne de la base de données et permet de faire évoluer l'un sans casser l'autre.

Mapper dédié

La conversion entre entités et DTOs est isolée dans des composants Spring dédiés (PlayerMapper, TournamentMapper), ce qui centralise cette logique de transformation plutôt que de la disperser dans les services ou les contrôleurs.

Gestion centralisée des erreurs (@RestControllerAdvice)

Chaque contrôleur est associé à un gestionnaire d'erreurs (PlayerControllerExceptionHandler, TournamentControllerExceptionHandler) qui intercepte les exceptions métier et les traduit en réponses HTTP cohérentes (404 pour une ressource introuvable, 400 pour une donnée invalide ou un conflit métier, 500 pour une erreur d'accès aux données), en s'appuyant sur un DTO d'erreur unique (Error).

Exceptions métier typées

Chaque cas d'erreur métier possède sa propre classe d'exception (PlayerNotFoundException, PlayerAlreadyExistsException, TournamentRegistrationException...), ce qui rend le code des services explicite et facilite le test unitaire de chaque scénario d'échec indépendamment des autres.

04 SYNTHÈSE DES COMPÉTENCES PROFESSIONNELLES MISES EN ŒUVRE

Ce projet backend a été conçu pour démontrer les quatre compétences professionnelles de l'activité type « Développer la partie back-end d'une application web ou web mobile sécurisée » du référentiel du titre professionnel Développeur Web et Web Mobile. Le tableau ci-dessous synthétise la mise en œuvre de chacune ; chaque compétence fait ensuite l'objet d'un chapitre détaillé.

CP	Compétence professionnelle	Mise en œuvre dans Instant Tennis
CP5	Mettre en place une base de données relationnelle	Conception d'un modèle de données relationnel (player, tournament, dyma_user, dyma_role et leurs tables d'association), déployé et versionné sur PostgreSQL via des migrations Flyway successives, avec contraintes d'intégrité (clés primaires, clés étrangères, unicité).
CP6	Développer des composants d'accès aux données SQL et NoSQL	Développement d'entités JPA (PlayerEntity, TournamentEntity, UserEntity) et de repositories Spring Data JPA encapsulant tous les accès SQL à PostgreSQL, avec requêtes dérivées sécurisées (sans risque d'injection).
CP7	Développer des composants métier côté serveur	Développement des contrôleurs REST et des services métier (PlayerService, TournamentService, RegistrationService, RankingCalculator) portant les règles de gestion, la validation des données et la sécurisation des accès (Spring Security, BCrypt, autorisation par rôle).
CP8	Documenter le déploiement d'une application dynamique web ou web mobile	Rédaction d'un guide de déploiement complet : conteneurisation Docker (image multi-stage, Docker Compose), configuration par profils dev/prod, documentation interactive de l'API avec Swagger UI / OpenAPI.

Remarque sur le périmètre de CP6 : la persistance de Instant Tennis repose exclusivement sur une base relationnelle (PostgreSQL). Aucun magasin NoSQL n'était nécessaire au regard des besoins fonctionnels du projet (données structurées uniquement, pas de fichiers volumineux ni de documents semi-structurés à stocker). La compétence est donc démontrée ici via son volet SQL ; le chapitre CP6 détaille néanmoins comment l'architecture en repositories dédiés permettrait d'intégrer un accès NoSQL supplémentaire (par exemple MongoDB) sans remettre en cause la couche métier, selon le même principe d'encapsulation.

05 CP5 — METTRE EN PLACE UNE BASE DE DONNÉES RELATIONNELLE

Compétence professionnelle n°5 du référentiel DWWM

La base de données relationnelle du projet a été conçue pour répondre aux besoins fonctionnels de gestion des joueurs, des tournois et des comptes utilisateurs. Le schéma est géré et versionné avec Flyway, ce qui garantit une évolution maîtrisée et reproductible de la structure de la base à chaque environnement (développement, test, production).

Entités du modèle

Entité	Description
player	Représente un joueur de tennis : identité, date de naissance, points et classement (rank).
tournament	Représente un tournoi : nom, dates de début et de fin, dotation (prize money) et capacité d'accueil.
player_tournament	Table d'association matérialisant l'inscription d'un joueur à un tournoi (relation plusieurs-à-plusieurs).
dyma_user	Représente un utilisateur de l'application (identifiants de connexion).
dyma_role	Référentiel des rôles applicatifs (ROLE_ADMIN, ROLE_USER).
dyma_user_role	Table d'association entre utilisateurs et rôles (relation plusieurs-à-plusieurs).
match	Représente une rencontre disputée entre deux joueurs au sein d'un tournoi (tour, score, vainqueur).
refresh_token	Jeton de rafraîchissement associé à un compte utilisateur, prévu pour la future authentification par JWT (voir chapitre Perspectives).

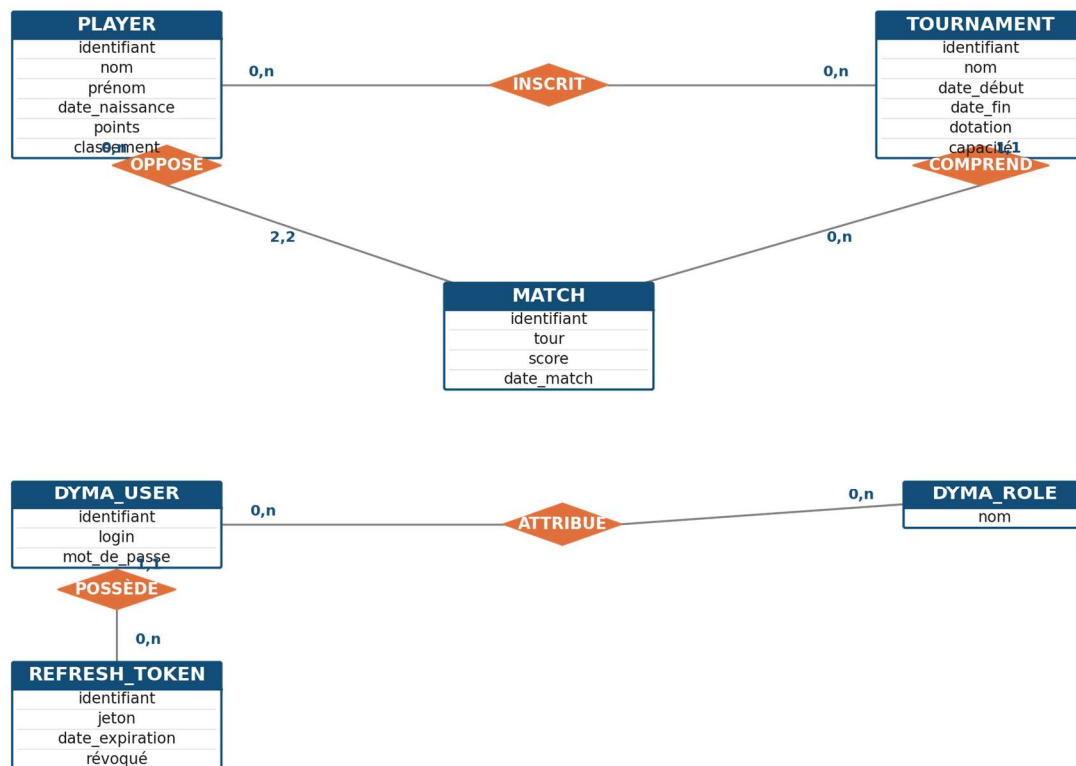
Les principales relations sont :

- un joueur (player) peut s'inscrire à plusieurs tournois et un tournoi peut accueillir plusieurs joueurs : relation plusieurs-à-plusieurs portée par la table player_tournament ;
- un utilisateur (dyma_user) peut se voir attribuer plusieurs rôles et un rôle peut être attribué à plusieurs utilisateurs : relation plusieurs-à-plusieurs portée par la table dyma_user_role.
- un tournoi (tournament) comprend plusieurs matchs (match), chaque match opposant exactement deux joueurs et pouvant désigner un vainqueur ;
- un utilisateur (dyma_user) peut posséder plusieurs jetons de rafraîchissement (refresh_token), chaque jeton appartenant à un seul utilisateur.

Modèle Conceptuel de Données (MCD)

Le modèle conceptuel identifie les entités du domaine métier, leurs propriétés et les associations qui les relient, indépendamment de toute implémentation technique. Il a été enrichi par rapport à la première version du dossier avec l'entité match (résultats de rencontres au sein d'un tournoi) et l'entité refresh_token, qui anticipe la bascule vers une authentification par JWT présentée au chapitre Perspectives d'évolution.

Modèle Conceptuel de Données (MCD) — Instant Tennis

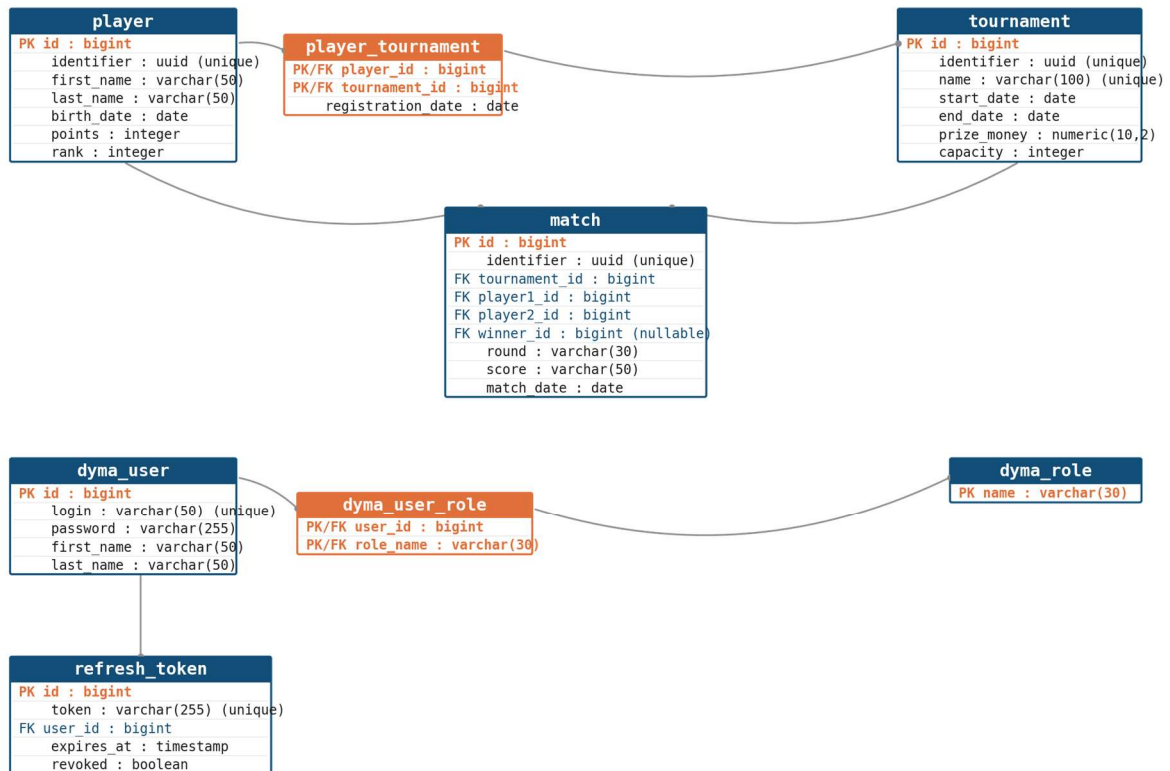


Modèle Conceptuel de Données (MCD) de Instant Tennis

Modèle Logique de Données (MLD)

Le modèle logique traduit le MCD en tables relationnelles : chaque entité devient une table, chaque association plusieurs-à-plusieurs devient une table de liaison portant les clés étrangères des deux entités reliées. Les clés primaires (PK) et étrangères (FK) ainsi que les types de données sont désormais explicités pour chacune des huit tables du schéma, en cohérence avec les entités JPA détaillées au chapitre CP6.

Modèle Logique de Données (MLD) — Instant Tennis



Modèle Logique de Données (MLD) de Instant Tennis

Migrations Flyway : construction incrémentale du schéma

Le schéma n'est jamais modifié à la main : chaque évolution est décrite dans un script SQL versionné (V001, V002, ...) placé dans `src/main/resources/db/migrations`. Flyway applique ces scripts dans l'ordre au démarrage de l'application et conserve un historique des migrations déjà exécutées, ce qui évite toute divergence entre les environnements.

Les migrations sont organisées en trois répertoires : `common` (structure et données communes à tous les environnements), `dev` (jeu de données de démonstration) et `prod` (jeu de données réel). Cette séparation, combinée aux profils Maven, permet de charger le bon jeu de scripts selon le profil actif (`spring.flyway.locations`).

```
-- V001__init_database.sql (extrait)
CREATE SEQUENCE player_id_seq;

CREATE TABLE player
(
    id integer NOT NULL DEFAULT nextval('player_id_seq'),
    last_name character varying(50) NOT NULL,
    first_name character varying(50) NOT NULL,
    birth_date date NOT NULL,
    points integer NOT NULL,
    rank integer NOT NULL,
    PRIMARY KEY (id)
);
```

Extrait de la migration Flyway initiale (V001__init_database.sql)

```
-- V007__make_player_identifieur_unique.sql
ALTER TABLE player
ALTER COLUMN identifieur SET NOT NULL;
```

```
ALTER TABLE player
ADD CONSTRAINT identifier_unique UNIQUE (identifier);

ALTER TABLE player
ADD CONSTRAINT player_unique UNIQUE (first_name, last_name, birth_date);
```

Ajout de contraintes d'intégrité de domaine sur l'entité player

Ces contraintes garantissent, au niveau de la base elle-même, qu'un identifiant public (UUID) est unique et qu'un même joueur (même prénom, nom et date de naissance) ne peut pas être enregistré deux fois — une double sécurité qui complète la validation applicative réalisée dans le PlayerService (voir chapitre CP7).

Table d'association et intégrité référentielle

```
-- V010__add_player_tournament_table.sql
CREATE TABLE player_tournament
(
    player_id bigint NOT NULL,
    tournament_id bigint NOT NULL,
    CONSTRAINT player_tournament_pkey PRIMARY KEY (player_id, tournament_id),
    CONSTRAINT fk_player FOREIGN KEY (player_id) REFERENCES public.player (id),
    CONSTRAINT fk_tournament FOREIGN KEY (tournament_id) REFERENCES public.tournament (id)
);
```

Table de liaison player_tournament portant la relation plusieurs-à-plusieurs

La clé primaire composite (player_id, tournament_id) empêche par construction qu'un même joueur soit inscrit deux fois au même tournoi, et les clés étrangères garantissent qu'aucune inscription ne peut référencer un joueur ou un tournoi inexistant.

Environnements de base de données

Trois configurations de base de données coexistent selon le profil actif : PostgreSQL 17 conteneurisé en développement et en production (voir CP8), et une base H2 en mémoire configurée en mode de compatibilité PostgreSQL (MODE=PostgreSQL) pour les tests automatisés, ce qui permet d'exécuter la suite de tests rapidement et de façon isolée, sans dépendance à un serveur PostgreSQL externe, tout en conservant une syntaxe SQL proche de la cible de production.

06 CP6 — DÉVELOPPER DES COMPOSANTS D'ACCÈS AUX DONNÉES SQL ET NOSQL

Compétence professionnelle n°6 du référentiel DWWM

L'accès aux données est entièrement encapsulé dans une couche dédiée (package data), qui est la seule autorisée à dialoguer avec la base de données. Ni les contrôleurs ni les services ne construisent de requête SQL : ils passent systématiquement par les repositories.

Entités JPA : mapping objet-relationnel

Les entités (PlayerEntity, TournamentEntity, UserEntity, RoleEntity) sont annotées @Entity et @Table et exposent un identifiant technique auto-généré (id) distinct de l'identifiant public exposé par l'API (identifier, de type UUID). Cette distinction protège la structure interne de la base : l'API ne révèle jamais la clé primaire technique, ce qui évite qu'un client puisse deviner ou énumérer les ressources par incrémentation d'identifiant.

```
@Entity
@Table(name = "player", schema = "public")
public class PlayerEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    @Column(name = "identifiant", nullable = false, unique = true)
    private UUID identifiant;

    @Column(name = "points", nullable = false)
    private Integer points;
    // ...
}
```

PlayerEntity : séparation entre identifiant technique (id) et identifiant public (identifiant)

Relation plusieurs-à-plusieurs : @ManyToMany et @JoinTable

La relation entre joueurs et tournois est représentée par une association @ManyToMany entre PlayerEntity et TournamentEntity. L'annotation @JoinTable décrit explicitement le nom de la table de liaison et ses colonnes de jointure, sans introduire de classe d'entité intermédiaire dédiée.

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "player_tournament",
    joinColumns = {@JoinColumn(name = "player_id", referencedColumnName = "id")},
    inverseJoinColumns = {@JoinColumn(name = "tournament_id", referencedColumnName = "id")}
)
private Set<TournamentEntity> tournaments = new HashSet<>();
```

Extrait de PlayerEntity : mapping de la relation plusieurs-à-plusieurs avec Hibernate

Repositories Spring Data JPA

L'accès aux données est encapsulé dans des interfaces qui étendent JpaRepository. Spring Data JPA génère automatiquement l'implémentation à partir de la signature des méthodes (query derivation), ce qui évite d'écrire manuellement le code d'accès aux données pour les requêtes courantes, tout en garantissant une liaison sécurisée des paramètres.

```
@Repository
public interface PlayerRepository extends JpaRepository<PlayerEntity, Long> {
```

```
Optional<PlayerEntity> findOneByIdentifier(UUID identifiant);

Optional<PlayerEntity> findOneByFirstNameIgnoreCaseAndLastNameIgnoreCaseAndBirthDate(
    String firstName, String lastName, LocalDate birthDate);
}
```

PlayerRepository : requêtes dérivées du nom de méthode (query derivation)

Spring Data JPA analyse le nom de la méthode `findOneByFirstNameIgnoreCaseAndLastNameIgnoreCaseAndBirthDate` pour générer automatiquement la requête JPQL correspondante, avec liaison sécurisée des paramètres (aucune concaténation de chaîne, donc aucune surface d'injection SQL).

Chaque ressource métier dispose de son propre repository, ce qui respecte le principe de responsabilité unique : `PlayerRepository` ne gère que les joueurs, `TournamentRepository` que les tournois, `UserRepository` que les comptes utilisateurs (avec une méthode dédiée `findOneWithRolesByLoginIgnoreCase` utilisée par la sécurité applicative, voir CP7).

Gestion des erreurs d'accès aux données

Chaque méthode de service qui sollicite un repository encapsule ses appels dans un bloc `try/catch` qui intercepte `DataAccessException` (hiérarchie d'exceptions Spring qui uniformise les erreurs JDBC/JPA) et la convertit en exception métier dédiée (`PlayerDataRetrievalException`, `TournamentDataRetrievalException`). Cette approche évite de laisser fuiter des détails techniques de la base de données dans les réponses de l'API.

```
try {
    return playerRepository.findAll().stream()
        .map(playerMapper::playerEntityToPlayer)
        .sorted(Comparator.comparing(player -> player.info().rank().position()))
        .collect(Collectors.toList());
} catch (DataAccessException e) {
    log.error("Could not retrieve players", e);
    throw new PlayerDataRetrievalException(e);
}
```

PlayerService.getAllPlayers() : encapsulation des erreurs d'accès aux données

Ouverture vers un accès NoSQL

Le projet Instant Tennis ne nécessite pas, dans son périmètre fonctionnel actuel, de stockage NoSQL : toutes les données manipulées (joueurs, tournois, utilisateurs) sont structurées et relationnelles par nature. La couche d'accès aux données est toutefois conçue de façon à pouvoir accueillir un magasin NoSQL supplémentaire sans impact sur la couche métier : il suffirait d'introduire un nouveau repository dédié (par exemple un `ImageRepository` basé sur Spring Data MongoDB, à l'image de ce qui se pratique pour le stockage de fichiers volumineux), consommé par un service applicatif au même titre que `PlayerRepository` ou `TournamentRepository`, sans modifier les contrôleurs existants. Cette capacité d'extension découle directement du principe d'encapsulation appliqué à chaque repository.

07 CP7 — DÉVELOPPER DES COMPOSANTS MÉTIER CÔTÉ SERVEUR

Compétence professionnelle n°7 du référentiel DWWM

Ce chapitre couvre l'ensemble des composants métier exécutés côté serveur : les contrôleurs REST qui exposent l'API, les services qui portent la logique de gestion, et les mécanismes de sécurisation et de validation qui protègent ces composants.

7.1 — Couche Web : les contrôleurs REST

Chaque ressource métier dispose de son propre contrôleur, annoté `@RestController` et `@RequestMapping`, qui expose les opérations CRUD standards sous forme d'endpoints REST documentés avec les annotations Swagger (`@Operation`, `@ApiResponse`s).

```
@Tag(name = "Tennis Players API")
@RestController
@RequestMapping("/players")
public class PlayerController {

    @Autowired
    private PlayerService playerService;

    @GetMapping
    public List<Player> list() {
        return playerService.getAllPlayers();
    }

    @PostMapping
    public Player createPlayer(@RequestBody @Valid PlayerToCreate playerToCreate) {
        return playerService.create(playerToCreate);
    }
}
```

Extrait de PlayerController : délégation immédiate au service, validation via @Valid

Points de terminaison exposés

Ressource	Méthode / URL	Rôle requis
Joueurs	GET /players	ROLE_USER
Joueurs	GET /players/{identifiant}	ROLE_USER
Joueurs	POST /players	ROLE_ADMIN
Joueurs	PUT /players	ROLE_ADMIN
Joueurs	DELETE /players/{identifiant}	ROLE_ADMIN
Tournois	GET /tournaments	ROLE_USER
Tournois	POST /tournaments	ROLE_ADMIN
Tournois	PUT /tournaments	ROLE_ADMIN
Tournois	DELETE /tournaments/{identifiant}	ROLE_ADMIN
Inscriptions	POST /tournaments/{tld}/players/{pld}	ROLE_ADMIN
Comptes	POST /accounts/login	public
Comptes	GET /accounts/logout	authenticifié

7.2 — Couche Métier : les composants Service

Les services encapsulent la logique de traitement de l'application. Ils orchestrent les repositories, appliquent les règles métier, gèrent les cas d'erreur et transforment les entités en DTOs via les mappers dédiés.

Exemple : création d'un joueur et recalcul du classement

La méthode `create` de `PlayerService` illustre bien cette orchestration : elle vérifie l'absence de doublon, enregistre le nouveau joueur, puis délègue à `RankingCalculator` le recalcul du classement de l'ensemble des joueurs avant de renvoyer le joueur nouvellement créé, avec son classement à jour.

```
public Player create(PlayerToCreate playerToCreate) {
    Optional<PlayerEntity> player = playerRepository
        .findOneByFirstNameIgnoreCaseAndLastNameIgnoreCaseAndBirthDate(
            playerToCreate.firstName(), playerToCreate.lastName(), playerToCreate.birthDate());
    if (player.isPresent()) {
        throw new PlayerAlreadyExistsException(
            playerToCreate.firstName(), playerToCreate.lastName(), playerToCreate.birthDate());
    }

    PlayerEntity playerToRegister = new PlayerEntity(
        playerToCreate.lastName(), playerToCreate.firstName(), UUID.randomUUID(),
        playerToCreate.birthDate(), playerToCreate.points(), 999999999);

    PlayerEntity registeredPlayer = playerRepository.save(playerToRegister);

    RankingCalculator rankingCalculator = new RankingCalculator(playerRepository.findAll());
    List<PlayerEntity> newRanking = rankingCalculator.getNewPlayersRanking();
    playerRepository.saveAll(newRanking);

    return this.getByIdentifier(registeredPlayer.getIdentifier());
}
```

Extrait de `PlayerService.create()` : contrôle de doublon, persistance, recalcul de classement

Le joueur est temporairement inséré avec un classement provisoire arbitraire (999999999) avant que `RankingCalculator` ne trie l'ensemble des joueurs par points décroissants et ne réattribue un rang cohérent à chacun. Cette classe est volontairement indépendante de Spring (pas d'annotation `@Component`) : elle ne fait que du calcul pur sur une liste en mémoire, ce qui la rend simple à tester unitairement sans contexte Spring.

Exemple d'orchestration métier : inscription d'un joueur à un tournoi

`RegistrationService` illustre un cas d'orchestration pure : il ne persiste pas directement de données propres, mais coordonne `PlayerRepository` et `TournamentRepository` pour vérifier trois règles de gestion avant d'autoriser l'inscription : l'existence du tournoi, sa capacité disponible, l'existence du joueur, puis l'absence de doublon d'inscription.

```
public void register(UUID tournamentIdentifier, UUID playerToRegister) {
    Optional<TournamentEntity> existingTournament =
        tournamentRepository.findOneByIdentifier(tournamentIdentifier);
    if (existingTournament.isEmpty()) {
        throw new TournamentRegistrationException(
            "Tournament " + tournamentIdentifier + " does not exist");
    }
    if (existingTournament.get().isFull()) {
        throw new TournamentRegistrationException(
            "Tournament " + existingTournament.get().getIdentifier() + " is full");
    }
    Optional<PlayerEntity> existingPlayer =
        playerRepository.findOneByIdentifier(playerToRegister);
    if (existingPlayer.isEmpty()) {
        throw new TournamentRegistrationException(
            "Player " + playerToRegister + " does not exist");
    }
}
```

```

    if (existingTournament.get().hasPlayer(existingPlayer.get())) {
        throw new TournamentRegistrationException(
            "Player " + playerToRegister + " is already registered to tournament " +
            tournamentIdentifier);
    }
    existingPlayer.get().addTournament(existingTournament.get());
    playerRepository.save(existingPlayer.get());
}

```

RegistrationService.register() : orchestration de deux repositories et contrôle de règles métier

7.3 — Sécurisation des composants métier

La sécurité applicative fait partie intégrante des composants métier côté serveur : elle protège chaque endpoint et chaque opération sensible, indépendamment de toute interface graphique.

Hachage des mots de passe (BCrypt)

Les mots de passe des utilisateurs ne sont jamais stockés en clair. Un PasswordEncoder basé sur l'algorithme BCrypt est déclaré comme bean Spring et utilisé aussi bien à la vérification des identifiants qu'à la création d'un compte.

```

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

```

Déclaration du PasswordEncoder dans SecurityConfiguration

Chargement des utilisateurs et de leurs rôles

Le service applicatif de sécurité implémente l'interface UserDetailsService fournie par Spring Security. Il recherche l'utilisateur par son login, puis convertit ses rôles métier (RoleEntity) en autorités Spring Security (GrantedAuthority) afin qu'elles puissent être évaluées par le mécanisme d'autorisation.

```

@Component
public class DymaUserDetailsService implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String login) throws UsernameNotFoundException {
        return userRepository.findOneWithRolesByLoginIgnoreCase(login)
            .map(this::createSecurityUser)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User with login " + login + " could not be found."));
    }

    private User createSecurityUser(UserEntity userEntity) {
        List<SimpleGrantedAuthority> grantedRoles = userEntity.getRoles().stream()
            .map(RoleEntity::getName)
            .map(SimpleGrantedAuthority::new)
            .toList();
        return new User(userEntity.getLogin(), userEntity.getPassword(), grantedRoles);
    }
}

```

Service applicatif de sécurité : pont entre le modèle métier (UserEntity, RoleEntity) et Spring Security

Authentification par session

La connexion est gérée par un endpoint explicite (AccountController) plutôt que par le formulaire de connexion par défaut de Spring Security, ce qui permet à un client REST (SPA, application mobile) de s'authentifier par un simple appel POST /accounts/login. L'AuthenticationManager valide les identifiants ; en cas de succès, le contexte de sécurité est explicitement enregistré dans la session HTTP.


```

@PostMapping("/login")
public void login(@RequestBody @Valid UserCredentials credentials,
                 HttpServletRequest request, HttpServletResponse response) {
    UsernamePasswordAuthenticationToken authenticationToken =
        new UsernamePasswordAuthenticationToken(credentials.login(), credentials.password());
    Authentication authentication = authenticationManagerBuilder.getObject()
        .authenticate(authenticationToken);
    SecurityContext securityContext = SecurityContextHolder.getContext();
    securityContext.setAuthentication(authentication);
    securityContextRepository.saveContext(securityContext, request, response);
}

```

AccountController.login() : authentification explicite et persistance du contexte de sécurité en session

Autorisation par rôle et par méthode HTTP

Le filtre de sécurité central (SecurityFilterChain) déclare, endpoint par endpoint et méthode HTTP par méthode HTTP, le rôle minimal requis pour y accéder. Les opérations de lecture sont ouvertes au rôle ROLE_USER, tandis que les opérations de création, modification et suppression sont réservées au rôle ROLE_ADMIN.

```

.authorizeHttpRequests(authorizations -> authorizations
    .requestMatchers("/swagger-ui/**").permitAll()
    .requestMatchers("/accounts/login").permitAll()
    .requestMatchers("/actuator/**").hasAuthority("ROLE_ADMIN")
    .requestMatchers(HttpMethod.GET, "/players/**").hasAuthority("ROLE_USER")
    .requestMatchers(HttpMethod.POST, "/players/**").hasAuthority("ROLE_ADMIN")
    .requestMatchers(HttpMethod.PUT, "/players/**").hasAuthority("ROLE_ADMIN")
    .requestMatchers(HttpMethod.DELETE, "/players/**").hasAuthority("ROLE_ADMIN")
    .anyRequest().authenticated()
);

```

Extrait de SecurityConfiguration : autorisation déclarative par méthode HTTP et par rôle

Durcissement des en-têtes HTTP

Au-delà de l'authentification, la configuration Spring Security applique plusieurs en-têtes de sécurité recommandés pour une API exposée sur Internet : une Content-Security-Policy restrictive, une interdiction d'affichage en iframe (frameOptions().deny(), qui protège du clickjacking) et une Permissions-Policy désactivant l'accès à la géolocalisation, au micro et à la caméra depuis le navigateur.

7.4 — Validation des données et gestion des erreurs

Avant toute persistance, les DTOs d'entrée sont systématiquement validés côté serveur grâce aux annotations Bean Validation, combinées à @Valid sur les contrôleurs. Cette validation ne peut pas être contournée par le client, contrairement à une validation qui ne serait effectuée que côté frontend.

```

public record PlayerToCreate(
    @NotBlank(message = "First name is mandatory") String firstName,
    @NotBlank(message = "Last name is mandatory") String lastName,
    @NotNull(message = "Birth date is mandatory")
    @PastOrPresent(message = "Birth date must be past or present") LocalDate birthDate,
    @PositiveOrZero(message = "Points must be more than zero") int points) {
}

```

PlayerToCreate : contraintes de validation portées directement par le record

En cas d'échec de validation, MethodArgumentNotValidException est interceptée par le gestionnaire d'erreurs global, qui construit une réponse HTTP 400 associant à chaque champ en erreur le message explicite défini dans l'annotation, facilitant le diagnostic côté client sans exposer de détail d'implémentation.

08 CP8 — DOCUMENTER LE DÉPLOIEMENT D'UNE APPLICATION WEB

Compétence professionnelle n°8 du référentiel DWWM

Ce chapitre constitue le guide de déploiement de Instant Tennis. Il décrit la conteneurisation Docker de l'application et de sa base de données, la configuration par profils, ainsi que les étapes concrètes pour construire, lancer et vérifier le bon fonctionnement de l'application, en local comme en production.

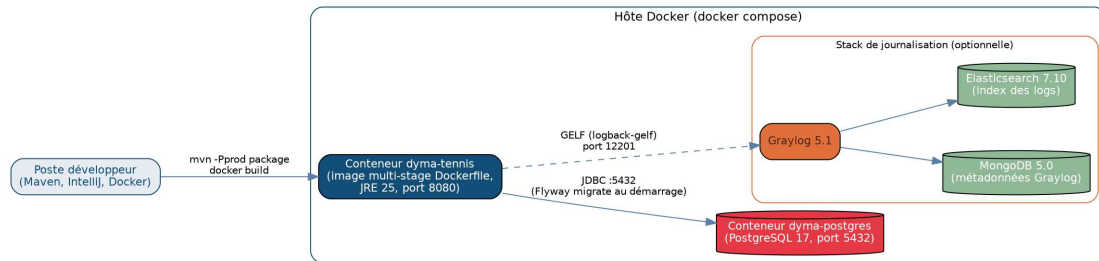


Schéma de déploiement conteneurisé de Instant Tennis

8.1 — Prérequis

- Docker et Docker Compose installés sur la machine cible ;
- Java 25 et Maven (uniquement nécessaires pour un build hors conteneur, à des fins de développement) ;
- un accès réseau au port 8080 (API) et 5432 (PostgreSQL) sur la machine hôte.

8.2 — Construction de l'image applicative : un Dockerfile multi-stage

Le Dockerfile utilise une construction en deux étapes (multi-stage build), une pratique recommandée pour produire une image de production légère et sans outillage de compilation superflu.

```
FROM maven:3.9.11-eclipse-temurin-25 as build
COPY pom.xml .
COPY src/ src/
RUN mvn -f pom.xml -Pprod clean package

FROM eclipse-temurin:25-jre as run
RUN useradd dyma
USER dyma
COPY --from=build /target/dyma-tennis.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Dockerfile multi-stage : build Maven isolé puis image d'exécution minimale

La première étape (build) embarque Maven et le JDK pour compiler l'application avec le profil prod. La seconde étape (run) repose sur une image beaucoup plus légère ne contenant qu'un JRE, dans laquelle seul le JAR exécutable produit à l'étape précédente est copié, ce qui réduit la taille de l'image finale et sa surface d'attaque. L'exécution avec un utilisateur dédié non privilégié plutôt qu'avec l'utilisateur root du conteneur constitue une bonne pratique de sécurité : en cas de compromission du processus applicatif, l'attaquant n'hérite pas de droits d'administration sur le système de fichiers du conteneur.

Commande de construction de l'image :

```
docker build -t instant-tennis:latest .
```

8.3 — Base de données conteneurisée

PostgreSQL est exécuté dans un conteneur dédié, décrit dans un fichier Docker Compose distinct par environnement, ce qui permet de faire varier le mot de passe, le port exposé et le nom du conteneur sans toucher au code de l'application.

```
services:
  dyma-postgresql:
    image: postgres:17
    container_name: dyma-postgres
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    ports:
      - "5432:5432"
    volumes:
      - dyma-postgres-data:/var/lib/postgresql/data

volumes:
  dyma-postgres-data:
```

docker/postgresql.yml : conteneur PostgreSQL avec volume nommé pour la persistance des données

Commande de démarrage de la base de données :

```
docker compose -f src/main/docker/postgresql.yml up -d
```

8.4 — Configuration par profils (dev / prod)

Le profil Maven actif (dev par défaut, prod sur demande) détermine, via un filtrage de ressource Maven, quel profil Spring Boot est activé au démarrage, ce qui charge le bon fichier `application-{profile}.properties` et donc le bon jeu de migrations Flyway et les bonnes informations de connexion à la base.

```
# application-dev.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/postgres
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.flyway.locations=classpath:db/migrations/common,classpath:db/migrations/dev

# application-prod.properties
spring.datasource.url=# prod config will be injected here #
spring.flyway.locations=classpath:db/migrations/common,classpath:db/migrations/prod
```

Externalisation de la configuration : les secrets de production sont injectés à l'exécution, jamais commités

En production, les informations de connexion réelles ne sont jamais stockées dans le dépôt de code : elles sont injectées au moment du déploiement (variables d'environnement ou secret manager), conformément aux bonnes pratiques de gestion des secrets.

8.5 — Lancement et vérification de l'application

Une fois l'image construite et la base de données démarrée, l'application se lance avec la commande suivante :

```
docker run --network=host -e SPRING_PROFILES_ACTIVE=dev instant-tennis:latest
```

La disponibilité du service peut ensuite être vérifiée via l'endpoint de santé exposé par Spring Boot Actuator, accessible sans authentification :

```
curl http://localhost:8080/healthcheck
```

8.6 — Documentation interactive de l'API (Swagger / OpenAPI)

Afin de faciliter la découverte, le test et l'intégration de l'API par un client tiers, l'application intègre springdoc-openapi, qui génère automatiquement une spécification OpenAPI 3 à partir du code annoté et l'expose via une interface Swagger UI interactive.

```
@Operation(summary = "Creates a player", description = "Creates a player")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Created player"),
    @ApiResponse(responseCode = "400", description = "Player already exists."),
    @ApiResponse(responseCode = "403", description = "This user is not authorized to perform
this action.")
})
@PostMapping
public Player createPlayer(@RequestBody @Valid PlayerToCreate playerToCreate) {
    return playerService.create(playerToCreate);
}
```

Documentation Swagger déclarative directement portée par le contrôleur

L'accès à `/swagger-ui/**` et `/v3/api-docs/**` est explicitement laissé public dans la configuration de sécurité, afin que la documentation reste consultable sans authentification, tandis que les appels réels vers les endpoints métier restent protégés par les règles d'autorisation présentées au chapitre CP7. Cette documentation interactive est accessible, une fois l'application démarrée, à l'adresse :

```
http://localhost:8080/swagger-ui/index.html
```

09 JOURNALISATION ET SUPERVISION

Au-delà des tests automatisés, deux dispositifs complémentaires renforcent la qualité et l'observabilité de l'application en fonctionnement : la traçabilité des requêtes SQL générées par Hibernate, et la centralisation des journaux applicatifs.

Traçabilité des requêtes SQL avec datasource-proxy

La dépendance `datasource-proxy-spring-boot-starter` intercepte chaque requête SQL exécutée par Hibernate et la journalise avec ses paramètres liés. En environnement de développement, cela permet de vérifier concrètement le SQL généré par Spring Data JPA pour une méthode donnée, de détecter d'éventuels problèmes de performance (requêtes N+1) et de s'assurer que les paramètres sont bien liés de façon préparée plutôt que concaténés.

Centralisation des journaux avec Graylog

La dépendance `logback-gelf` permet à l'application d'exporter ses journaux au format GELF (Graylog Extended Log Format) vers un serveur Graylog, au lieu de se limiter à une sortie console locale. Une stack complète (MongoDB pour les métadonnées, Elasticsearch pour l'indexation, Graylog pour l'interface d'exploration) peut être démarrée indépendamment de l'application pour centraliser et rechercher les journaux de plusieurs environnements depuis une interface unique.

Supervision applicative avec Spring Boot Actuator

Spring Boot Actuator expose des endpoints de supervision (health, metrics), restreints au rôle `ROLE_ADMIN` dans la configuration de sécurité. L'endpoint `/healthcheck` reste public afin de permettre à un orchestrateur (ou à un simple docker healthcheck) de vérifier la disponibilité du service sans nécessiter d'authentification.

10 PLAN DE TESTS, JEU D'ESSAI ET COUVERTURE DE CODE

Afin de garantir la fiabilité et la non-régression de l'application, une stratégie de tests à plusieurs niveaux a été mise en place, couvrant la logique métier isolée, le comportement des contrôleurs et le parcours complet de bout en bout, base de données comprise.

Environnement de tests

- Frameworks : JUnit 5, Mockito et AssertJ pour les assertions fluides ; MockMvcTester pour les tests de la couche Web ; TestRestTemplate pour les tests de bout en bout.
- Isolation : les tests unitaires de service simulent les repositories avec Mockito (@Mock, @ExtendWith(MockitoExtension.class)), sans jamais solliciter de base de données réelle.
- Base de données de test : les tests d'intégration et de bout en bout s'exécutent sur une base H2 en mémoire configurée en mode PostgreSQL, avec un jeu de migrations Flyway dédié au profil test, nettoyée et remigrée avant chaque test (flyway.clean() / flyway.migrate()).
- Sécurité désactivée en test : un profil Spring dédié (@Profile("test")) remplace la configuration de sécurité réelle par une chaîne de filtres qui autorise toutes les requêtes, afin que les tests de bout en bout se concentrent sur la logique métier sans avoir à gérer l'authentification.

Niveaux de tests réalisés

Tests unitaires de service (Mockito)

PlayerServiceTest, TournamentServiceTest et RegistrationServiceTest vérifient la logique métier en isolation complète, en simulant le comportement des repositories. Ils couvrent notamment le cas nominal (récupération, création) et les cas d'erreur (donnée introuvable, échec d'accès aux données).

```
@Test
public void shouldFailToReturnPlayersRanking_WhenDataAccessExceptionOccurs() {
    Mockito.when(playerRepository.findAll())
        .thenReturn(new DataRetrievalFailureException("Data access error"));

    Assertions.assertThatThrownBy(() -> playerService.getAllPlayers())
        .assertInstanceOf(PlayerDataRetrievalException.class)
        .hasMessage("Could not retrieve player data");
}
```

PlayerServiceTest : vérification qu'une erreur d'accès aux données est bien traduite en exception métier

Tests de la couche Web (@WebMvcTest)

PlayerControllerTest et TournamentControllerTest chargent uniquement le contexte Spring MVC nécessaire au contrôleur testé, le service métier étant remplacé par un mock (@MockitoBean). Ces tests vérifient la sérialisation JSON des réponses et la traduction des exceptions métier en codes HTTP corrects.

```
@Test
public void shouldReturn404NotFound_WhenPlayerDoesNotExist() {
    UUID playerToRetrieve = UUID.fromString("aaaaaaaa-1111-2222-3333-bbbbbbbbbbbb");
    Mockito.when(playerService.getByIdentifier(playerToRetrieve))
        .thenReturn(new PlayerNotFoundException(playerToRetrieve));

    var response = mockMvc.get().uri("/players/aaaaaaaa-1111-2222-3333-bbbbbbbbbbbb")
        .accept(MediaType.APPLICATION_JSON).exchange();

    var json = response.assertThat().hasStatus(HttpStatus.NOT_FOUND).bodyJson();
    json.extractingPath("$.errorDetails")
        .isEqualTo("Player with identifier aaaaaaaaa-1111-2222-3333-bbbbbbbbbbbb could not be found.");
}
```

Tests de bout en bout (@SpringBootTest, TestRestTemplate)

PlayerControllerEndToEndTest, TournamentControllerEndToEndTest et les tests d'intégration des services démarrent l'application complète sur un port aléatoire et exécutent de véritables requêtes HTTP contre une base H2 réinitialisée à chaque test. Ils valident le comportement réel de bout en bout, y compris le recalcul de classement après suppression d'un joueur.

```
@Test
public void shouldDeletePlayer() {
    restTemplate.delete("/players/d27aef45-51cd-401b-a04a-b78a1327b793");

    ResponseEntity<List<Player>> allPlayersResponseEntity = restTemplate.exchange(
        "/players", HttpMethod.GET, null,
        new ParameterizedTypeReference<List<Player>>() {});

    Assertions.assertThat(allPlayersResponseEntity.getBody()
        .extracting("info.lastName", "info.rank.position")
        .containsExactly(Tuple.tuple("NadalTest", 1), Tuple.tuple("FedererTest", 2)));
}
```

PlayerControllerEndToEndTest : la suppression déclenche bien le recalcul du classement des joueurs restants

Couverture de code (JaCoCo)

La couverture de code est mesurée avec JaCoCo, intégré au build Maven via le plugin jacoco-maven-plugin, qui instrumente le bytecode pendant l'exécution des tests (mvn test) et génère un rapport HTML détaillé par classe, méthode et branche.

```
<!-- Extrait de pom.xml -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

Configuration JaCoCo ajoutée au pom.xml pour générer le rapport de couverture à chaque build

Le rapport est généré localement après exécution de la commande mvn test, puis consultable dans target/site/jacoco/index.html. Le tableau ci-dessous reprend la structure du rapport JaCoCo (par package) ; les valeurs sont à relever depuis le rapport HTML généré sur le poste de développement au moment de la soutenance.

Package	Couverture d'instructions	Couverture de branches
com.dyma.tennis.service	à relever depuis target/site/jacoco/	à relever depuis target/site/jacoco/
com.dyma.tennis.web	à relever depuis target/site/jacoco/	à relever depuis target/site/jacoco/
com.dyma.tennis.data	à relever depuis target/site/jacoco/	à relever depuis target/site/jacoco/
Total projet	à relever depuis target/site/jacoco/	à relever depuis target/site/jacoco/

Jeu d'essai — création d'un joueur

Le tableau suivant synthétise les scénarios de test appliqués à la création d'un joueur (POST /players), en s'appuyant sur les contraintes de validation portées par le record PlayerToCreate et sur la règle métier d'unicité vérifiée par PlayerService.

Scénario	Donnée en entrée	Résultat attendu	Résultat
Cas nominal	Prénom, nom, date de naissance passée, points ≥ 0	HTTP 200, joueur créé avec un classement calculé	✓
Nom manquant	lastName = null	HTTP 400, message "Last name is mandatory"	✓
Date de naissance future	birthDate = date future	HTTP 400, message "Birth date must be past or present"	✓
Points négatifs	points = -100	HTTP 400, message "Points must be more than zero"	✓
Joueur déjà existant	Même prénom, nom et date de naissance qu'un joueur existant	HTTP 400, PlayerAlreadyExistsException	✓
Identifiant inconnu (GET/PUT/DELETE)	identifiant aléatoire non présent en base	HTTP 404, PlayerNotFoundException	✓

Jeu d'essai — inscription à un tournoi

Scénario	Contexte	Résultat attendu	Résultat
Inscription nominale	Tournoi existant, non complet ; joueur existant, non encore inscrit	HTTP 200, joueur inscrit au tournoi	✓
Tournoi introuvable	tournamentIdentifier inconnu	HTTP 400, "Tournament ... does not exist"	✓
Tournoi complet	Nombre de joueurs inscrits = capacité du tournoi	HTTP 400, "Tournament ... is full"	✓
Joueur introuvable	playerIdentifier inconnu	HTTP 400, "Player ... does not exist"	✓
Double inscription	Le joueur est déjà inscrit à ce tournoi	HTTP 400, "Player ... is already registered..."	✓

L'ensemble de ces scénarios est automatisé et rejoué à chaque build via Maven (mvn test), ce qui garantit la non-régression du comportement métier à chaque évolution du code.

11 MICROSERVICES : ARCHITECTURE CIBLE

Instant Tennis est aujourd'hui un monolithe modulaire : un seul module Maven, une seule application déployée, mais des packages internes déjà séparés par responsabilité (web, service, data, security — voir chapitre 03). Cette structure est volontaire et adaptée au périmètre actuel du projet. Elle constitue également une base saine pour une évolution vers une architecture microservices, qui est présentée ici comme cible d'évolution à moyen terme plutôt que comme un existant.

L'objectif d'une telle décomposition n'est pas de multiplier les services pour eux-mêmes, mais de répondre à des besoins concrets qui apparaîtraient avec la montée en charge du produit : déployer et faire évoluer indépendamment les composants, isoler les pannes, et externaliser la configuration et la découverte de services plutôt que de les coder en dur. Quatre microservices ont été retenus pour cette cible, ce qui reste volontairement simple et démontrable :

Microservice	Rôle
webapp	Cœur métier de Instant Tennis : reprend les couches web / service / data actuelles (PlayerController, PlayerService, PlayerRepository...) sous forme d'un service autonome, packagé et déployé indépendamment.
consul	Annuaire de services (service discovery) et supervision : chaque instance de webapp s'enregistre auprès de Consul au démarrage et expose son état de santé, ce qui permet un routage dynamique et une détection automatique des instances défaillantes.
config-server	Serveur de configuration centralisée (Spring Cloud Config) : externalise les propriétés applicatives (URL de base de données, profils, secrets non sensibles) dans un dépôt Git versionné, distribué à chaud aux microservices qui le consomment.
dyma-postgresql	Base de données relationnelle dédiée : le conteneur PostgreSQL 17 déjà utilisé en CP8 devient le magasin de persistance attitré du microservice webapp, avec sa propre gestion de volume et de sauvegardes.

11.1 — webapp : le cœur métier

Le microservice webapp concentre l'ensemble de la logique actuellement développée dans ce dossier (CP5 à CP8) : contrôleurs REST, services métier, repositories Spring Data JPA et configuration de sécurité. Le découpage en couches déjà en place (chapitre 03) rend cette extraction directe : aucune classe ne dépend d'un état partagé en dehors de son package, ce qui limite le travail de bascule à des questions de configuration et de packaging plutôt qu'à une réécriture du code métier. Au démarrage, webapp interroge config-server pour récupérer sa configuration puis s'enregistre auprès de consul afin d'être détectable par les autres composants de l'écosystème (une future passerelle API, par exemple).

11.2 — consul : annuaire de services et supervision

Consul assure deux fonctions complémentaires. La première est la découverte de services (service discovery) : plutôt que de coder en dur l'adresse réseau d'une instance de webapp, les clients internes interrogent Consul pour obtenir la liste des instances saines disponibles à un instant donné, ce qui permet de faire évoluer le nombre d'instances (montée en charge horizontale) sans reconfiguration manuelle. La seconde est la supervision active : Consul interroge périodiquement l'endpoint de santé déjà exposé par Spring Boot Actuator (voir chapitre 09) et retire automatiquement de l'annuaire toute instance qui ne répond plus, ce qui évite de router du trafic vers un service défaillant.

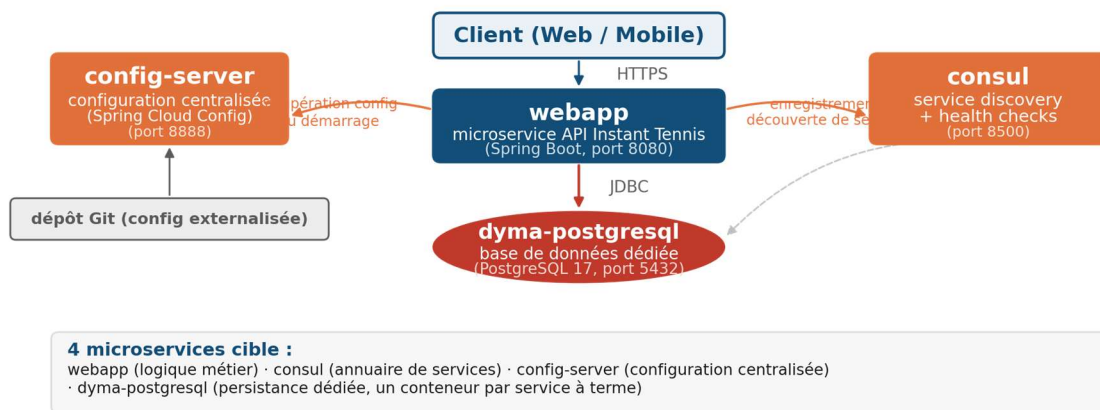
11.3 — config-server : configuration centralisée

Aujourd'hui, la configuration de Instant Tennis repose sur des profils Maven et des fichiers application-{profil}.properties embarqués dans le livrable (chapitre CP8). Cette approche fonctionne bien pour un service unique, mais devient difficile à maintenir dès que plusieurs instances ou plusieurs services doivent partager et faire évoluer les mêmes propriétés. config-server, basé sur Spring Cloud Config, résout ce problème en

centralisant la configuration dans un dépôt Git versionné : chaque microservice récupère sa configuration à distance au démarrage via un simple appel HTTP, ce qui permet de modifier un paramètre pour tous les environnements sans reconstruire les images Docker, et de conserver un historique auditable de chaque changement de configuration.

11.4 — dyma-postgresql : persistance dédiée

La base de données conserve le rôle qu'elle a déjà dans ce dossier (CP5, CP8) : un conteneur PostgreSQL 17 versionné par les migrations Flyway existantes. Dans la cible microservices, ce conteneur est explicitement associé au seul microservice webapp, ce qui respecte le principe « une base par service » recommandé par les architectures microservices : aucun autre composant (consul, config-server) n'accède directement à cette base, ce qui évite les couplages implicites et permet de faire évoluer le schéma sans impact sur le reste de l'écosystème.



Architecture microservices cible de Instant Tennis (webapp, consul, config-server, dyma-postgresql)

Cette cible reste compatible avec la conteneurisation déjà mise en place (chapitre CP8) : consul, config-server et dyma-postgresql peuvent être ajoutés au fichier Docker Compose existant au même titre que le conteneur webapp, chacun comme un service à part entière, ce qui permet une adoption progressive sans remise en cause de l'existant.

12 PERSPECTIVES D'ÉVOLUTION

Ce dossier documente l'état actuel du backend de Instant Tennis, conçu et développé pour répondre au périmètre défini avec le formateur (chapitre 01). Plusieurs axes d'évolution ont néanmoins été identifiés au fil du développement ; ils sont présentés ici à titre prospectif, sans avoir été implémentés dans la version actuelle du projet.

12.1 — Sécurité avancée : authentification par JWT

L'application repose actuellement sur une authentification par session (chapitre CP7) : après connexion, le contexte de sécurité est conservé côté serveur et associé à un cookie de session. Ce mécanisme est simple et parfaitement adapté à un client REST unique, mais il impose un état serveur (session) qui complique la scalabilité horizontale évoquée au chapitre Microservices — une requête doit alors être routée vers l'instance qui détient la session, ou celle-ci doit être partagée entre instances.

Une évolution naturelle consiste à remplacer ce mécanisme par une authentification par jeton JWT (JSON Web Token) : après connexion, le serveur délivre un jeton signé contenant l'identité et les rôles de l'utilisateur, que le client renvoie ensuite dans l'en-tête Authorization de chaque requête. L'authentification devient alors totalement stateless côté serveur — n'importe quelle instance de webapp peut valider le jeton sans interroger un état partagé — ce qui correspond exactement au besoin de scalabilité horizontale de l'architecture microservices cible. La table `refresh_token`, déjà ajoutée au modèle de données (chapitre CP5), anticipe cette évolution : elle permettrait de délivrer des jetons d'accès de courte durée de vie associés à un jeton de rafraîchissement plus long, révocable indépendamment, pour limiter l'impact d'un jeton compromis sans dégrader l'expérience utilisateur.

12.2 — Monétisation pour les visiteurs du site

Le périmètre actuel du dossier ne couvre pas de modèle économique : l'API est exposée sans distinction commerciale entre utilisateurs. Plusieurs pistes de monétisation orientées visiteurs pourraient néanmoins être envisagées pour un produit destiné à être exploité au-delà du cadre de la formation :

- un modèle freemium sur l'accès à l'API : les endpoints de consultation (classement, calendrier des tournois) resteraient gratuits, tandis que des fonctionnalités avancées (statistiques détaillées, export de données, alertes personnalisées) seraient réservées à un compte payant ;
- un abonnement pour les organisateurs de tournois, qui donnerait accès à des quotas plus élevés (nombre de tournois créés, nombre de joueurs inscrits) et à des outils de gestion supplémentaires, sur le modèle SaaS B2B ;
- un espace de mise en avant payant pour des partenaires (équipementiers, clubs) sur les pages de classement ou de tournoi, dans le respect d'une charte de contenu qui resterait à définir.

Techniquement, ces pistes s'appuieraient naturellement sur le modèle de rôles déjà en place (`dyma_role`) : il suffirait d'introduire un rôle `ROLE_PREMIUM` au même titre que `ROLE_USER` et `ROLE_ADMIN`, puis de conditionner l'accès à certains endpoints via les mêmes règles d'autorisation déclaratives déjà utilisées dans `SecurityConfiguration` (chapitre CP7), sans remettre en cause l'architecture existante.

12.3 — Pipeline d'intégration et de déploiement continu (CI/CD)

Le projet réunit déjà les prérequis techniques d'une chaîne CI/CD (chapitre 10 et conclusion) : une suite de tests automatisés à trois niveaux exécutable via `mvn test`, une mesure de couverture avec JaCoCo, et une image Docker reproductible construite par un Dockerfile multi-stage. L'axe d'évolution le plus directement exploitable consiste donc à assembler ces briques dans un pipeline automatisé (GitHub Actions, par exemple) déclenché à chaque livraison de code : exécution des tests et du rapport de couverture, construction de l'image Docker en cas de succès, puis publication de cette image vers un registre de conteneurs.

Cette évolution présente un double intérêt pédagogique et opérationnel : elle sécurise chaque changement de code par une vérification automatique avant toute mise en production, et elle constitue le prolongement logique de l'architecture microservices cible (chapitre 11), où le déploiement fréquent et indépendant de chaque service devient la norme plutôt que l'exception.

13 CONCLUSION

Le développement du backend de Instant Tennis a permis de mettre en pratique les quatre compétences professionnelles attendues d'un Développeur Web et Web Mobile pour l'activité type « Développer la partie back-end d'une application web ou web mobile sécurisée ».

La mise en place d'une base de données relationnelle versionnée par migrations Flyway (CP5), le développement de composants d'accès aux données encapsulés dans des repositories Spring Data JPA (CP6), la construction de composants métier côté serveur sécurisés et validés (CP7), ainsi que la documentation complète du déploiement conteneurisé de l'application (CP8), constituent les quatre piliers démontrés par ce projet.

La couverture par les tests automatisés, à trois niveaux complémentaires (unitaire, contrôleur, bout en bout), ainsi que la conteneurisation Docker de l'application et de sa base de données, préparent le terrain pour une intégration dans une chaîne d'intégration et de déploiement continu (CI/CD).

Ce travail constitue une étape clé dans la validation du titre professionnel de Développeur Web et Web Mobile et renforce la capacité à mener la conception et le développement d'une application backend professionnelle de bout en bout.

Remerciements

Je tiens à remercier mes formateurs pour leur accompagnement tout au long de cette formation, pour la qualité de leurs explications techniques et pour leur patience face à mes nombreuses questions ; leur exigence et leurs retours ont largement contribué à la qualité de ce projet.

Je remercie également mes camarades de promotion pour leur entraide, les échanges techniques et la bonne humeur partagée pendant ces mois de formation, qui ont rendu cet apprentissage aussi agréable qu'exigeant.

Enfin, je remercie les membres du jury pour le temps qu'ils consacrent à l'évaluation de ce dossier et de la soutenance qui l'accompagne.